EXP 1

1.

```java
import java.util.*;

class Solution {
    public static int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }

    public static void main(String[] args) {
        int[] arr = {10, 5, 4, 3, 48, 6, 2, 33, 53, 10};
        int k = 4;

        System.out.println(kthSmallest(arr, k));
    }
}
```

2.

```java
import java.util.Arrays;

class Solution {
    int getMinDiff(int[] arr, int k) {
        int n = arr.length;

        if (n == 1) {
            return 0;
        }

        Arrays.sort(arr);

        int ans = arr[n - 1] - arr[0];
        int smallest = arr[0] + k;
        int largest = arr[n - 1] - k;

        for (int i = 1; i < n; i++) {
            if (arr[i] - k < 0) {
                continue;
            }

            int currentMin = Math.min(smallest, arr[i] - k);
            int currentMax = Math.max(largest, arr[i - 1] + k);

            ans = Math.min(ans, currentMax - currentMin);
        }

        return ans;
    }
}
```

3.

```java
import java.util.Arrays;

class Solution {
    public void mergeArrays(int[] a, int[] b) {
        int i = a.length - 1;
        int j = 0;

        while (i >= 0 && j < b.length) {
            if (a[i] > b[j]) {
                int temp = a[i];
                a[i] = b[j];
                b[j] = temp;
                i--;
                j++;
            } else {
                break;
            }
        }

        Arrays.sort(a);
        Arrays.sort(b);
    }
}
```

4.

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length <= 1) {
            return intervals;
        }

        java.util.Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        java.util.List<int[]> result = new java.util.ArrayList<>();
        int[] currentInterval = intervals[0];
        result.add(currentInterval);

        for (int[] interval : intervals) {
            if (currentInterval[1] >= interval[0]) {
                currentInterval[1] = Math.max(currentInterval[1], interval[1]);
            } else {
                currentInterval = interval;
                result.add(currentInterval);
            }
        }

        return result.toArray(new int[result.size()][]);
    }
}
```

5.

```java
class Solution {
    public java.util.List<Integer> commonElements(java.util.List<Integer> arr1, java.util.List<Integer> arr2, java.util.List<Integer> arr3) {
        java.util.List<Integer> result = new java.util.ArrayList<>();
        int i = 0, j = 0, k = 0;

        while (i < arr1.size() && j < arr2.size() && k < arr3.size()) {
            int val1 = arr1.get(i);
            int val2 = arr2.get(j);
            int val3 = arr3.get(k);

            if (val1 == val2 && val2 == val3) {
                result.add(val1);

                while (i < arr1.size() && arr1.get(i) == val1) i++;
                while (j < arr2.size() && arr2.get(j) == val1) j++;
                while (k < arr3.size() && arr3.get(k) == val1) k++;
            } else if (val1 < val2) {
                i++;
            } else if (val2 < val3) {
                j++;
            } else {
                k++;
            }
        }

        return result;
    }
}
```

6.

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length <= 1) {
            return intervals;
        }

        java.util.Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        java.util.List<int[]> result = new java.util.ArrayList<>();
        int[] currentInterval = intervals[0];
        result.add(currentInterval);

        for (int[] interval : intervals) {
            if (currentInterval[1] >= interval[0]) {
                currentInterval[1] = Math.max(currentInterval[1], interval[1]);
            } else {
                currentInterval = interval;
                result.add(currentInterval);
            }
        }

        return result.toArray(new int[result.size()][]);
    }
}
```

7.

```java
class Solution {
    public java.util.List<Integer> commonElements(java.util.List<Integer> arr1, java.util.List<Integer> arr2, java.util.List<Integer> arr3) {
        java.util.List<Integer> result = new java.util.ArrayList<>();
        int i = 0, j = 0, k = 0;

        while (i < arr1.size() && j < arr2.size() && k < arr3.size()) {
            int val1 = arr1.get(i);
            int val2 = arr2.get(j);
            int val3 = arr3.get(k);

            if (val1 == val2 && val2 == val3) {
                result.add(val1);

                while (i < arr1.size() && arr1.get(i) == val1) i++;
                while (j < arr2.size() && arr2.get(j) == val1) j++;
                while (k < arr3.size() && arr3.get(k) == val1) k++;
            } else if (val1 < val2) {
                i++;
            } else if (val2 < val3) {
                j++;
            } else {
                k++;
            }
        }

        return result;
    }
}
```

8.

```java
import java.util.ArrayList;
import java.util.Collections;

class Solution {
    static ArrayList<Integer> factorial(int N) {
        ArrayList<Integer> res = new ArrayList<>();
        res.add(1);
        for (int x = 2; x <= N; x++) {
            int carry = 0;
            for (int i = 0; i < res.size(); i++) {
                int prod = res.get(i) * x + carry;
                res.set(i, prod % 10);
                carry = prod / 10;
            }
            while (carry > 0) {
                res.add(carry % 10);
                carry /= 10;
            }
        }
        Collections.reverse(res);
        return res;
    }
}
```

9.

```java
import java.util.HashMap;

class Solution {
    public boolean isSubset(int[] a, int[] b) {
        HashMap<Integer, Integer> count_a = new HashMap<>();
        for (int x : a) {
            count_a.put(x, count_a.getOrDefault(x, 0) + 1);
        }
        for (int x : b) {
            if (count_a.getOrDefault(x, 0) > 0) {
                count_a.put(x, count_a.get(x) - 1);
            } else {
                return false;
            }
        }
        return true;
    }
}
```

10.

```java
class Solution {
    public int maxWater(int arr[]) {
        if (arr == null || arr.length == 0) return 0;
        int n = arr.length;
        int l = 0, r = n - 1;
        int l_max = 0, r_max = 0;
        int res = 0;
        while (l <= r) {
            if (arr[l] <= arr[r]) {
                if (arr[l] >= l_max) l_max = arr[l];
                else res += l_max - arr[l];
                l++;
            } else {
                if (arr[r] >= r_max) r_max = arr[r];
                else res += r_max - arr[r];
                r--;
            }
        }
        return res;
    }
}
```